

Guía de cierre y rúbrica del examen suspenso

Equipo BCEL — AcadTrace, gestión académica escolar con bitácora de auditoría encadenada

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/acadtrace
Cabecera evaluada	Rama main, commit 78b0d175 — 720 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Bedón Viteri Keyla Betzabe · Castro López Pedro Leonardo · Emanuel Pino Juliana Romina · Luna Mora Ernesto Gregory
Rinden el examen suspenso	Los cuatro integrantes
Complejidad del proyecto	40 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: al corte el repositorio aún estaba en LE023as/acadtrace; se transfirió después y al 16/09 figura bajo [gleiston-guerrero/acadtrace](https://github.com/gleiston-guerrero/acadtrace). Cumple si sigue así al corte del examen. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/acadtrace>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	70 %	Véase §1
2	Estructura del repositorio, secretos y archivos de raíz	Por culminar	Por culminar	40 %	Véase §2
3	Línea base congelada e integración	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	50 %	Véase §4
5	Patrones GoF y ADR	Por culminar	Por culminar	30 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	55 %	Véase §6
7	Calidad de la aplicación web	Por culminar	Por culminar	20 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	70 %	Véase §8
9	Calidad de la aplicación móvil	Por culminar	Por culminar	35 %	Véase §9
10	Integración y contratos	Por culminar	Por culminar	30 %	Véase §10

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
11	Persistencia distribuida	Por culminar	Por culminar	25 %	Véase §11
12	Pirámide de pruebas y cadena de auditoría	Por culminar	Por culminar	35 %	Véase §12
13	Pipeline CI/CD	Por modificar	Por modificar	60 %	Véase §13
14	Observabilidad	Por culminar	Por culminar	40 %	Véase §14
15	Protocolo y campaña experimental	Por culminar	Por culminar	25 %	Véase §15
16	Paquete de datos y reproducibilidad	Por culminar	Por culminar	35 %	Véase §16
17	Evaluación ISO/IEC 25010	Por culminar	Por culminar	20 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por culminar	55 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	50 %	Véase §19
20	Trabajo en equipo y autoría	Por culminar	Por culminar	35 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

Entre el 12/09 y el corte cerraron trabajo real. Los 29 commits que estaban sin fusionar están en `main`. La cabeza de la cadena de auditoría tiene ahora bloqueo efectivo en los tres escritores (`@Lock(PESSIMISTIC_WRITE)` y `FOR UPDATE`) con el reloj de Lamport persistido, y las tres implementaciones de la forma canónica usan los mismos 11 campos: una cadena de cuatro eslabones generada en Java se verifica con el verificador de Python y la manipulación se detecta. Finales de línea normalizados (38 CSV sin CR y comprobación en CI que puede fallar); `npm ci` con lockfiles sincronizados en las cuatro interfaces; sonda de salud del despliegue que propaga el fallo; configuración de gitleaks sin lista blanca; cinco imágenes publicadas en GHCR por commit; Soporte con 73,03 % de instrucciones y compuerta de módulo; seis pruebas Playwright bien diseñadas (fallan sin entorno y verifican escritura y restauración); verificación de bytes 6/6 con diez pruebas que detectan manipulación. La app móvil guarda el JWT en `EncryptedSharedPreferences`, tiene biometría y caché Room sin conexión. El manuscrito compila sin referencias indefinidas.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando

a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica en README, CITATION.cff, manuscrito, OpenAPI y despliegue; integrantes, correos y roles	1	1	2	1	5	1,8	Por modificar	0,72
§2a	Credenciales y datos personales fuera del árbol y del historial, claves rotadas, sin valores de reserva ni acceso dev-token, escaneo de secretos como compuerta	3	3	3	3	12	4,3	Por culminar	0,86
§2b	Sin informes de cobertura, logs, fotos, APK duplicados ni lockfile huérfano; un solo protocolo, juego de resultados y tablero	1	1	3	2	7	2,5	Por modificar	1,00
§2c	.env.example alineado con compose, propiedades y SPA	1	1	1	1	4	1,4	Por modificar	0,56
§3	Rama principal protegida, PR aprobados por otro integrante, tabla de revisión real, CHANGELOG.md y etiqueta anotada de cierre	2	1	2	2	7	2,5	Por culminar	0,50
§4	Dominio sin JPA ni Spring, puertos en domain/port y aplicación dependiente sólo de puertos en Principal y Secretaría; ArchUnit	4	4	5	5	18	6,6	Por modificar	2,64
§5	Facade, Template Method, Observer y Strategy invocados desde flujos reales con pruebas; ADR-005 y ADR-006 veraces	3	3	3	4	13	4,7	Por culminar	0,94
§6	SPA con /main, sesión fuera de localStorage, validación de expiración, llamadas sólo por el gateway con autorización y portal de directivo	3	3	4	4	14	5,1	Por modificar	2,04
§7	TypeScript estricto, ESLint en CI sin errores, Vitest con cobertura ≥ 70 % y Dockerfile multi-stage para cada SPA	3	3	4	4	14	5,1	Por culminar	1,02
§8	Actualización por arrastre, push demostrable de extremo a extremo y URL base por BuildConfig con HTTPS	2	2	2	2	8	2,9	Por modificar	1,16
§9	Pruebas de los ViewModels, E2E instrumentada en CI y APK release firmado y publicado por CI	3	3	3	3	12	4,3	Por culminar	0,86
§10	Contratos generados por la SPA y la app Kotlin, verificados contra Principal y Docente con estados reales; API de calificaciones autenticada	3	4	4	4	15	5,4	Por culminar	1,08

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§11	Clúster de base de datos reproducible con healthchecks, Flyway con validación, usuario de ejecución sin privilegios y caída de nodo medida	4	4	3	4	15	5,4	Por culminar	1,08
§12a	Cadena de auditoría: vectores canónicos compartidos, restricción UNIQUE, prueba multiservicio real y fallos que no se pierdan en silencio	5	4	4	4	17	6,2	Por modificar	2,48
§12b	Cobertura $\geq 70\%$ por módulo sin clases elegidas, E2E en verde como compuerta y Testcontainers con migraciones completas	3	4	5	5	17	6,2	Por culminar	1,24
§12c	Rampa 0→200 real, un juego canónico de carga, cifras inexistentes retiradas y comandos del README ejecutables	3	2	3	3	11	4,0	Por culminar	0,80
§13	Compuertas en los needs del despliegue, jobs que hacen lo que su nombre dice, despliegue por imagen y CI sin secretos de producción	3	3	2	3	11	4,0	Por modificar	1,60
§14	Métricas de negocio en flujos reales, registros JSON con trace_id en los cuatro servicios, OpenTelemetry con Collector y tablero único con evidencia	3	3	4	4	14	5,1	Por culminar	1,02
§15	Protocolo con H0/H1 y umbrales versionado antes de medir y campaña medida sobre el sistema real (no por fórmula) con $r = 10$	4	3	3	4	14	5,1	Por culminar	1,02
§16	Generación determinista que reproduce los datos publicados, verificador corregido, manifiesto completo, diccionario y regeneración en CI	4	3	3	3	13	4,7	Por culminar	0,94
§17	ISO/IEC 25010 con las cinco características de la guía, réplicas reales, IC95 % y tabla generada	4	2	2	3	11	4,0	Por culminar	0,80
§18	Manuscrito sin afirmaciones que el código no respalde, PDF igual a la fuente, ≥ 12 referencias citadas con DOI/ISBN y trazabilidad E1→E4	3	2	3	3	11	4,0	Por culminar	0,80
§19	Amenazas con mitigaciones verificables (incluidos datos sintéticos) y reflexión ética $\geq \frac{1}{2}$ página con ACM 2018	2	1	1	2	6	2,2	Por modificar	0,88
§20	Actas contemporáneas sin anacronismos, declaración de IA única, identidades git correctas y aporte propio de cada integrante	2	1	2	2	7	2,5	Por culminar	0,50
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **2,65 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00. Los cuatro integrantes rinden el examen: el aporte de cada uno se acredita sólo con sus propios commits y PR sobre ítems de esta guía (§20).

3. Trabajo exigido

§1 Identificación del entregable y URL

Qué encontré: la URL canónica no aparece en ningún archivo; LE023as/acadtrace figura en nueve lugares (README, CITATION.cff, portada y cuerpo del manuscrito, docs/api/openapi.yaml, el documento de rotación de secretos, docs/bitacora.md, que además apunta a otro repositorio, y el job de despliegue, que sigue clonando el repositorio antiguo). CITATION.cff empieza con BOM, tiene un correo de autor distinto del institucional y declara version: 4.0.0 sin etiqueta que le corresponda. El README no tiene integrantes ni roles. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/acadtrace> en todos esos lugares; los cuatro integrantes con nombre completo, correo institucional y rol en README y CITATION.cff; versión igual a la etiqueta de cierre. **Cómo lo verifico:** `git grep -n "LE023as/" <etiqueta> vacío; head -c3 CITATION.cff | xxd sin efbbbf.`

§2 Estructura — credenciales, datos personales y basura versionada

a) Secretos y datos sensibles (lo más urgente de esta guía). Corrigieron la lista blanca de gitleaks y los \${VAR} de Principal y Soporte, pero al corte:

- docs/pdf/MANUAL_MAESTRO_COMANDOS_OPERACIONES_SGA.pdf contiene, en texto extraíble, la contraseña de PostgreSQL, la dirección del servidor, el nombre de la clave SSH y cuentas de usuario con contraseña; ningún detector de texto inspecciona PDF.
- El historial no se purgó: los valores reales del secreto JWT, la contraseña de base de datos, la clave AES y el testigo gRPC siguen en decenas de commits alcanzables.
- La migración V18 (13/09) crea el rol sga_app con una contraseña literal, repetida en dos pruebas.
- microservicio-secretaria/.../application.properties define valores de reserva para JWT, AES, base de datos y gRPC: si falta la variable, el servicio firma testigos con una clave conocida; y docker-compose.yml perdió los :? obligatorios.
- JwtAuthFilter.java de Secretaría concede los roles SECRETARIA y DIRECTOR a **cualquier** testigo que empiece por dev-token, y la interfaz de Secretaría tiene un botón visible “Ingresar con los 4 Roles” que lo usa, sin condición de entorno de desarrollo.
- scripts/backups/backup_pre_seed_20260806_135036.sql contiene 677 registros de estudiantes con cédula, nombres y correo, y 21 hashes de contraseña; el 92 % de las cédulas supera el dígito verificador. Mientras no se demuestre que son sintéticos, deben tratarse como datos personales de menores.
- El job secret-scan sólo revisa el rango del evento y ningún job depende de él.

Cómo debe quedar: ni credenciales ni datos personales en el árbol (incluidos PDF, PPTX y SQL) ni en el historial (git filter-repo coordinado), o bien declaración con evidencia fechada (sin los valores) de que todas las credenciales publicadas se rotaron; ningún valor de reserva para secretos; V18 sin contraseña literal (marcador de Flyway); la rama startsWith("dev-token") y el botón eliminados; gitleaks detect --log-opts="--all" como compuerta en los needs de la publicación. **Cómo lo verifico:**

```
gitleaks detect --source . --log-opts="--all" # 0 hallazgos
git grep -nI -E "PASSWORD '[^$]|dev-token" -- ':!*md' # vacío
for f in $(git ls-files '*.pdf'); do pdftotext "$f" - | grep -HiE "pass:|password|.pem"; done # vacío
git ls-files scripts/backups # vacío
```

y una petición con Authorization: Bearer dev-token-x a Secretaría devuelve 401.

b) Basura versionada. 823 informes HTML de cobertura (13 MB) y otra copia en Secretaría; build.log, maven-debug.log y otros logs pese a que .gitignore ya los excluye; fotos subidas; dos

APK de depuración; un `package-lock.json` de 102 bytes sin `package.json`; 18 rutas de evidencias con espacios; dos protocolos casi idénticos, dos carpetas de resultados y dos tableros distintos. **Deben:** sacarlos del índice, publicar informes como artefactos de CI y dejar un solo protocolo, un solo juego de resultados y un solo tablero. **c)** `.env.example`. Compose exige `GRAFANA_ADMIN_PASSWORD` y la plantilla define otra variable, así que `docker compose up` falla con la plantilla copiada; faltan `MAIL_PASSWORD`, `FIREBASE_CREDENTIALS_PATH`, `ZIPKIN_ENDPOINT`, `CORS_ORIGIN`, `DB_SSL`, `UPLOADS_PATH`, `VITE_API_URL` y `VITE_API_PRINCIPAL`, y `DB_HOST` apunta al servidor real. **Cómo lo verifico:** `git ls-files | grep -cE "\.log$|docs/cobertura/uploads/"` da 0; `cp .env.example .env && docker compose config -q` termina con 0.

§3 Línea base — revisión cruzada que no existió

Qué encontré: `pre-e4` es ligera y `v1.0.0` es ligera y posterior al corte; no hay `CHANGELOG`. Los 26 PR fusionados entre el 12 y el 13/09 tienen **cero revisiones**, la mayoría fusionados por su autor; los PR que el manuscrito presenta como “validados por otro integrante” (#48, #51, #55, #58, #59, #60) también tienen cero, y el #58 lo abrió y fusionó una persona distinta de la que la tabla indica. No existe ningún issue, aunque el manuscrito describe un flujo “formalizado mediante un issue”. 184 de 271 commits de la primera línea de `main` entraron directamente. Una fusión sin revisión (62933d72, PR #98) dejó el `application.properties` de Secretaría en 0 bytes durante unas 17 horas. **Cómo debe quedar:** protección de `main` (PR obligatorio, una aprobación de otro integrante, verificaciones obligatorias); tabla de revisión cruzada del manuscrito sustituida por los datos reales de la API (PR, autor, quien fusiona, quien aprueba) o eliminada, igual que la descripción del flujo por issues; ramas residuales borradas; `CHANGELOG.md` con entradas fechadas E1–E4 y examen suspenso; etiqueta anotada de cierre sobre el último commit de `main`, creada **al final de todo**. **Cómo lo verifico:**

```
git cat-file -t v4.0.0 && test "$(git rev-parse v4.0.0~{commit})" = "$(git rev-parse origin/main)"
git ls-remote --tags origin v4.0.0 # deben salir DOS líneas
```

y `/pulls/<n>/reviews` con `APPROVED` de un usuario distinto del autor en cada PR del suspenso.

§4 Backend en capas — el dominio de Principal es JPA

Qué encontré: en `sga-principal`, 27 de 74 archivos de `domain` llevan `@Entity`, `@Table` y `@Column` (y dos estrategias llevan `@Component`); no hay ningún puerto en el dominio; 17 archivos de `application` importan `infrastructure` y los servicios usan directamente 25 `JpaRepository`. Entre el 12/09 y el corte se añadió otra entidad JPA al dominio (`EstadoCadenaAuditoria`). En Secretaría el “dominio” son sólo DTO y 15 servicios usan `JdbcTemplate` directamente; Docente (Django) no tiene capa de dominio. Sólo Soporte tiene un puerto con su adaptador. **Cómo debe quedar:** en Principal y Secretaría, dominio sin `jakarta.persistence` ni `org.springframework`, puertos en `domain/port`, entidades JPA y adaptadores con mapeadores en `infrastructure/persistence`, aplicación que dependa sólo de puertos y prueba `ArchUnit`; en Docente, capa de servicios con puertos explícitos y un ADR que justifique la adaptación a Django. **Cómo lo verifico:**

```
grep -rLE "jakarta.persistence|org.springframework" sga-principal/src/main/java/ec/edu/uteq/sga/domain | wc -l
↪ # 0
grep -rL "import ec.edu.uteq.sga.infrastructure" sga-principal/src/main/java/ec/edu/uteq/sga/application | wc
↪ -1 # 0
```

§5 Patrones GoF — cuatro de cinco son clases que nadie llama

Qué encontré: sólo `Repository` se usa. `PortalAcademicoFacade` tiene **cero** llamadores; `ReporteNotasPeriodoPDF` (`Template Method`) tiene cero y genera texto, no PDF; `NotaPublicadaEvent` sólo lo publica la fachada muerta y su oyente escribe en el log “Simulación del despacho” (el push real de Docente no sigue el patrón); `CalculoPromedioStrategy` sólo la usa la fachada muerta,

y el promedio real está fijo (70/30) en `microservicio-docente/docentes/services.py`. ADR-005 cita `CalificacionService` y `CalificacionRepository`, que no existen, no tiene alternativas ni consecuencias negativas y es anterior a las clases. ADR-006 está desactualizado (dice que no se integra FCM y cita 36 pruebas cuando hay 62) y sus criterios no tienen fuente externa. **Cómo debe quedar:** el registro real de calificaciones pasa por la fachada desde un controlador; el promedio se calcula con la estrategia seleccionada por la configuración del año lectivo; un endpoint de reporte por período usa la plantilla y genera PDF; `NotaPublicadaEvent` se publica desde el registro real y su oyente invoca el servicio de push; una prueba de integración por patrón; ADR-005 con rutas verificables, alternativas y consecuencias negativas; ADR-006 actualizado con dos criterios cuantitativos con fuente. **Cómo lo verifico:**

```
for c in PortalAcademicoFacade ReporteNotasPeriodoPDF CalculoPromedioStrategy NotaPublicadaEvent; do
  echo $c $(grep -rl $c sga-principal/src/main/java | grep -v "/$c.java" | grep -ci "controller\|service");
  ↪ done # >= 1 cada uno
```

§6 Aplicación web — el testigo en `localStorage` y un manuscrito que dice lo contrario

Qué encontré: existen `/`, `/login`, `/settings` y `/about`, `i18n` y tema en `sga-frontend` y Secretaría; **falta** `/main`. Las cuatro SPA guardan el JWT en `localStorage` plano, lo que la guía prohíbe expresamente, y `ProtectedRoute` sólo comprueba que exista un testigo, sin validar `exp`; el manuscrito afirma “sin `localStorage`, cookies `HttpOnly` y `SameSite`” y “React + TypeScript” (hay 33 `.jsx`, ningún `.ts` ni `tsconfig`). `Calificaciones.jsx` envía las notas de los estudiantes por HTTP a una dirección fija del servicio de IA, saltando el gateway y **sin cabecera de autorización**; `Reportes.jsx` de Secretaría llama a `localhost:3000`. No hay portal de directivo propio. **Cómo debe quedar:** una SPA canónica con las cinco rutas; testigo en memoria con renovación, `sessionStorage` con expiración o cookie `HttpOnly` emitida por el backend; guarda que valide `exp`; todas las llamadas por el gateway con autorización; vista de directivo con reporte por período; el manuscrito describiendo exactamente eso. **Cómo lo verifico:** `grep -rn "localStorage.*token" sga-principal/sga-frontend/src | wc -l` da 0; `grep -n 'path="/main"' sga-principal/sga-frontend/src/App.jsx`; ninguna dirección IP ni `localhost` en el código de las SPA.

§7 Calidad web — sin tipos, sin lint en CI y sin una sola prueba ejecutada

Qué encontré: las cuatro interfaces son JavaScript puro; `npx eslint .` en `sga-frontend` da 54 errores y 9 avisos, y el job `lint` no ejecuta ESLint (sólo un subconjunto de `flake8`); no hay Vitest ni Jest configurado y el único archivo de prueba web no lo ejecuta ningún script; `test-web` sólo compila. `sga-frontend` y `docente-frontend` no tienen `Dockerfile` y en `compose` corren con `npm run dev`, es decir, el servidor de desarrollo en despliegue. El artefacto `web-dist` por commit sí existe, sin suma. **Cómo debe quedar:** `sga-frontend` en TypeScript con `strict: true` y `tsc --noEmit` en verde; ESLint con cero errores en el job `lint`; Vitest con Testing Library y cobertura ≥ 70 % sobre todo `src/` con `thresholds` que hagan fallar `test-web`; `Dockerfile` multi-stage (Node para construir, `nginx:alpine` para servir) para cada SPA desplegada, usado en `compose` y en la publicación; suma del artefacto `dist`. **Cómo lo verifico:**

```
cd sga-principal/sga-frontend && npm ci && npx tsc --noEmit && npx eslint . && npx vitest run --coverage #
↪ código 0
grep -c "^FROM" sga-principal/sga-frontend/Dockerfile #
↪ >= 2
```

§8 Aplicación móvil

Qué encontré: inicio de sesión restringido a representantes, listado, detalle con calificaciones y asistencia, comunicados, caché `Room`, `SyncWorker`, biometría y servicio FCM registrado. Faltan la actualización por arrastre (sólo hay un botón “Reintentar”); el push no es operativo tal como está el repositorio (sin `google-services.json` y con Firebase deshabilitado en el backend) y no hay evidencia de recepción; la URL base es una dirección fija por HTTP con `usesCleartextTraffic="true"`. **Cómo**

debe quedar: PullToRefreshBox en listado y detalle; instrucciones para suministrar las credenciales de Firebase fuera del repositorio y evidencia fechada de una notificación recibida tras publicar una nota (captura y registro del backend); URL base por BuildConfig con HTTPS o tráfico sin cifrar limitado a depuración. **Cómo lo verifico:** `grep -rn PullToRefresh app-movil-docente/app/src/main;` ninguna dirección IP en `app/src/main`.

§9 Calidad móvil

Qué encontré: 62 pruebas unitarias, pero sólo un ViewModel de 13 está probado (`LoginViewModel` no); la única instrumentada renderiza una pantalla y el CI no la ejecuta. El APK versionado es de depuración, del 04/09, y ni siquiera contiene la mensajería FCM que entró el 09/09; el CI sólo hace `assembleDebug`; al corte no había release firmado. **Cómo debe quedar:** pruebas de los ViewModels de inicio de sesión, listado, detalle y comunicados; una E2E instrumentada con servidor simulado (inicio de sesión, listado, detalle) ejecutada en CI con emulador; APK release firmado con almacén en secretos, generado y publicado por CI con su SHA-256 e idéntico al de `release/apk/`. **Cómo lo verifico:** `grep -n "connectedAndroidTest\\|android-emulator-runner\\|assembleRelease" .github/workflows/ci-cd.yml; apksigner verify --print-certs release/apk/*.apk` sin “Android Debug”.

§10 Contratos — una interacción, con el proveedor simulado

Qué encontré: un único contrato con una interacción (`GET /api/docente/calificaciones/`), cuyo “consumidor” está escrito en Python dentro del propio proveedor y cuya verificación parchea `get_queryset`: sin base de datos ni estados. No hay contrato de la app móvil ni de ninguna SPA, ni contra Principal o Secretaría. `tests/contract/test_contracts.py` son expresiones regulares sobre `.proto` y OpenAPI. Además `CalificacionViewSet` no tiene `permission_classes` ni hay permisos por defecto: **la API de calificaciones de Docente no exige autenticación.** **Cómo debe quedar:** contratos generados por `sga-frontend` (Pact-JS sobre su módulo de servicios) y por la app Kotlin (sobre `RepresentanteApi`), que cubran inicio de sesión, representados, calificaciones y asistencia; verificación contra Principal y Docente con estados que siembren una base de prueba; `IsAuthenticated` en los `ViewSet` con un contrato que cubra el 401; job de contratos en los `needs` de la publicación. **Cómo lo verifico:** `ls pacts/*.json ≥ 3; grep -rn patch.object microservicio-docente/tests/contract vacío; petición sin testigo a la API de calificaciones devuelve 401.`

§11 Persistencia distribuida — no hay clúster

Qué encontré: `docker-compose.yml` define 15 servicios y **ninguno es de base de datos**; todos apuntan a un único PostgreSQL externo. El manuscrito afirma “tres instancias redundantes ...Raft” y particionado por rango, que no existen en el código. La evidencia de tolerancia a fallos mide la conmutación de réplicas de la aplicación detrás de HAProxy, no de la base de datos. Flyway tiene la validación desactivada en `compose`, V13 se modificó después de aplicarse, hay un hueco en V11 y dos `v5_`; Secretaría no ejecuta Flyway. V18 crea `sga_app` con privilegios reducidos, pero **todos los servicios siguen conectándose como** `postgres`, de modo que la revocación no tiene efecto, y la misma migración concede `DELETE` sobre la tabla que guarda la cabeza de la cadena. **Cómo debe quedar:** clúster reproducible en `compose` (tres nodos CockroachDB, o primario y réplicas PostgreSQL) con `healthchecks` y `depends_on: service_healthy`; Flyway con validación activa y cambios de V13 en una migración nueva; usuario de migración separado del de ejecución (`sga_app`) en los tres backends, sin `DELETE` sobre la cadena; guion que detiene un nodo con escrituras en curso y verifica continuidad e integridad de la cadena, con su registro fechado versionado. **Cómo lo verifico:** `grep -c healthcheck docker-compose.yml ≥ 3; grep -rn VALIDATE_ON_MIGRATE=false docker-compose.yml vacío; ningún DB_USER de ejecución igual a postgres.`

§12 Pruebas y cadena de auditoría

a) Cadena de auditoría. Sigue habiendo tres implementaciones que difieren en casos límite: la clave `contraseña` está escrita en Java con un byte inválido (`"contrase?a"`), de modo que no se excluye y su valor entra en el JSON; `BigDecimal 9.50, 1e20` y las marcas de tiempo se serializan distinto en Java y en Python. Ninguna prueba verifica con Python filas escritas por Java; el reloj vectorial sigue sin efecto; no hay restricción `UNIQUE` que impida la bifurcación; las pruebas de concurrencia no usan las migraciones reales; `guardar` atrapa `Exception` y sólo registra un log, así que **un fallo de auditoría pierde el evento en silencio**; ADR-007 y el documento de alineación contradicen el código. **Deben:** vectores canónicos compartidos (ñ y tildes, decimales, flotantes, fechas, nulos, claves excluidas) que las tres implementaciones pasen en CI; índice único sobre `hash_anterior`; prueba de integración con migraciones reales y escrituras concurrentes desde los tres servicios; fallo de auditoría que aborte la transacción o quede en una cola persistente; ADR-007 reescrito; reloj vectorial con vector recibido real o retirado del diseño.

b) Cobertura, E2E y Testcontainers. Medida sobre todo el código escrito a mano: sga-principal 24,98 % de instrucciones, agregado Java 49,02 %. Las compuertas del 70 % siguen aplicándose a seis y siete clases elegidas a mano (el “90,7 %” del manuscrito sale del agregado de esas siete), y el PR #118 “para alcanzar la meta del 70 %” excluye lógica real (`SecurityConfig`, `GlobalExceptionHandler`, el servicio gRPC escrito a mano) y sube Secretaría de 69,76 % a 70,46 % de líneas. Docente cuenta sus propios archivos de prueba en el denominador. Los HTML de cobertura del 13/09 se editaron a mano en vez de regenerarse. La E2E de navegador estaba **en rojo en el commit de corte** y aun así se desplegó porque no está en los `needs`; modifica calificaciones del entorno desplegado; sus evidencias son de antes de reescribirla. Las suites con contenedores se omiten sin Docker, sólo una aplica migraciones reales y parte de un esquema V1–V8 escrito a mano, y la de Secretaría traga los errores de `migrate()` en un `catch` vacío. **Deben:** regla `BUNDLE` de 70 % (líneas e instrucciones) sin `includes` y excluyendo sólo código generado; pruebas hasta alcanzarla en Principal; `.coveragerc` que omita las pruebas en Docente; un único número por módulo con fecha y comando en README y manuscrito; E2E en verde en el commit de cierre, dentro de los `needs`, contra un entorno de pruebas y con su informe publicado; Testcontainers con migraciones completas que fallen (no se omitan) en CI.

c) Carga y comandos. No hay una rampa válida (el juego “de estrés” llega a 200 usuarios a los 40s y otro se quedó en 1 usuario). El manuscrito conserva filas ISO duplicadas con “0 fallos en 12.265” (cifra que no está en ningún CSV de ningún commit) y “100 % de fallos en 12.735”; cita ocho hashes de commit que no existen; el certificado de Soporte da 0 de 4. Ocho comandos del README fallan en un clon limpio (falta instalación previa, dependencias no declaradas, un `.tex` inexistente, guiones sin permiso de ejecución). **Deben:** rampa con `LoadTestShape 0→200` lineal en 600s; un juego canónico por escenario y los demás en `experimentos/descartados/` con su motivo; cifras y hashes corregidos; README ejecutable de principio a fin. **Cómo lo verifico:**

```
grep -n "<element>CLASS</element>\\|<include>" sga-principal/pom.xml microservicio-secretaria/backend/pom.xml
↪ # vacio
grep -c "12{,}265\\|12{,}735" Informe-E4_BCEL/TA-PFC-E4_BCEL.tex
↪ # 0
grep -rn "contrase?a" --include=*.java . | wc -l
↪ # 0
git ls-files -s scripts/start.sh sga-principal/mvnw app-movil-docente/gradlew | awk '{print $1}' | sort -u
↪ # 100755
```

§13 Pipeline CI/CD — compuertas fuera del camino del despliegue

Qué encontré: sonda de salud efectiva, cinco imágenes por commit e instalación reproducible. Pero `secret-scan` y la E2E no están en ningún `needs` (con la E2E en rojo se desplegó igual) y `build-images` construye en otras ramas aunque fallen las pruebas; `lint` no ejecuta el linter de ninguna SPA, `test-web` no ejecuta pruebas, `integration` es un despliegue por SSH sin pruebas de integración y el APK es sólo de depuración; una fusión reintrodujo una publicación duplicada de

Secretaría; el despliegue reconstruye en el servidor con `docker compose up --build` en lugar de usar las imágenes publicadas; y `test-backend` inyecta la dirección, el usuario `postgres` y la contraseña de la base de datos real. **Cómo debe quedar:** despliegue que dependa de `secret-scan`, E2E, contratos y todas las pruebas; jobs que hacen lo que su nombre dice (§7, §9); integración que levante el stack en el runner, espere salud y ejecute `tests/integration` y contratos antes de desplegar; una sola publicación por imagen, incluidas las SPA; despliegue con `docker compose pull` de las imágenes del commit; ningún secreto de producción en pruebas. **Cómo lo verifico:** lectura de los `needs` de `integration`; `grep -n "DB_HOST" .github/workflows/ci-cd.yml` sin la dirección real; todos los jobs en verde en el commit de cierre.

§14 Observabilidad — métricas de negocio que nadie incrementa

Qué encontré: `SgaBusinessMetrics` define seis métricas `sga_*` sin ningún llamador, y su indicador de estudiantes activos está fijo en 344; Docente sí incrementa las suyas. Registros JSON con `trace_id` sólo en Secretaría y Docente. No hay OpenTelemetry ni Collector (el manuscrito lo deja como trabajo futuro); sólo Soporte envía a Zipkin. Hay dos tableros distintos, ninguno usa `sga_*` ni `docente_*`, y `prometheus.yml` conserva un `remote_write` de plantilla. La captura “bajo carga” es del 31/08, anterior a la corrida oficial y al tablero actual. **Cómo debe quedar:** métricas de negocio incrementadas en flujos reales (nota registrada, matrícula confirmada, notificación enviada, evento de auditoría encadenado) y métricas del motor del clúster; registros JSON con `trace_id` en los cuatro servicios; agente o SDK OpenTelemetry hacia un Collector en `compose` y de ahí a un almacén de trazas; un solo `ops/grafana/pfc-dashboards.json` provisionado con los seis paneles de la guía; captura fechada del tablero y de una traza distribuida durante la corrida canónica.

§15 Campaña experimental — latencias generadas por fórmula

Qué encontré: dos protocolos casi idénticos, sin H0/H1 ni variables, con 30 réplicas en lugar de $r = 10$ con descarte y una cuenta errónea ($30 \times 4 \times 5 = 120$); el manuscrito afirma un descarte de la primera y la última repetición que no existe en el código. Los CSV entraron el 31/08 y los protocolos el 01/09, ya con “métricas obtenidas” dentro: **los umbrales se fijaron después de medir**. En el código del corte, `run_experimentos.py` (modo local, en proceso, sin base de datos ni HTTP) **genera las latencias del experimento 1 con una fórmula** (`base_net = 1.25 + 0.35 * conc` más constantes por mecanismo y ruido) y fija `detectado = False` para M0 y M1, en contradicción con `REPRODUCIBILIDAD.md`. Réplicas degeneradas: 47 filas de `exp1_concurrencia.csv` idénticas salvo el número de réplica, `throughput` 20.000 = 20/0,001, 2 filas distintas de 60 en `exp3`. Los CSV no tienen marcas de tiempo. Ninguna campaña se ejecutó contra los servicios desplegados. **Cómo debe quedar:** un único `experimentos/protocolo-e4.md` con H0/H1, variables, umbrales numéricos, $r = 10$ con descarte implementado y procedimiento de manipulación sobre la base real, versionado en un commit anterior a los datos; campaña en modo HTTP o sobre la base del clúster, con latencias medidas con `perf_counter_ns()` alrededor de escrituras reales, marca de tiempo por réplica y cada réplica como archivo crudo. **Cómo lo verifico:**

```
grep -n "base_net\|detectado = False" experimentos/run_experimentos.py      # vacío
git log --format=%ct -1 -- experimentos/protocolo-e4.md                  # menor que:
git log --format=%ct --diff-filter=A -1 -- experimentos/resultados/raw
```

§16 Paquete de datos — el certificado verifica bytes, no reproducibilidad

Qué encontré: `sha256sum -c` da 6/6, pero regenerando dos veces con el código del corte **los resultados cambian entre ejecuciones** y, salvo `iso25010.csv`, ninguno coincide con el certificado. M3 da `reconciliacion_automatica=False` y el CSV versionado dice `True`. **La detección de M2 y M3 cae del 100 % al 60 %:** las manipulaciones T2 (`payload`) y T5 (marca de tiempo) no se detectan porque el verificador sólo hasha el contenido canónico; ese cambio entró el 13/09 después de los CSV y del certificado. Los datos publicados no salen del código publicado, y puede haber una

brecha real en el verificador. Además el guion escribe fuera de `--output-dir`, el manifiesto cubre 6 archivos (fuera quedan Locust, falsos positivos, HTML y figuras), no hay diccionario de datos, `requirements.txt` sólo declara `matplotlib` y el CI nunca regenera. **Cómo debe quedar:** verificador que incluya `payload` y marca de tiempo (o justificación de por qué T2 y T5 no aplican); generación determinista con semilla en todas las fuentes y figuras sin metadatos variables; escritura sólo en `--output-dir`; un solo juego canónico; manifiesto de todos los crudos; `DICCIONARIO_DATOS.md`; dependencias y versión de Python completas; job de CI que regenere en un directorio temporal y compare con el manifiesto. **Cómo lo verifico:**

```
python experimentos/run_experimentos.py --mode local --output-dir /tmp/a
python experimentos/run_experimentos.py --mode local --output-dir /tmp/b
diff <(cd /tmp/a && sha256sum *) <(cd /tmp/b && sha256sum *)          # sin diferencias
diff <(cd /tmp/a && sha256sum *) <(cd experimentos/resultados && sha256sum $(ls /tmp/a))  # sin diferencias
git status --porcelain docs/                                       # vacío
```

§17 Evaluación ISO/IEC 25010

Qué encontré: la tabla del manuscrito no tiene media, DE ni IC95 %, duplica “Disponibilidad” y “Fiabilidad” con cifras contradictorias, omite Compatibilidad y no mide la tasa de 5xx durante una hora. `iso25010.csv` no contiene réplicas: son 20 ventanas de dos corridas no canónicas (una de ellas con un usuario y 79 % de fallos), con cobertura constante de 30,31 % de un informe antiguo; la tabla usa un juego y el CSV otro. Los intervalos de detección corresponden a $n = 100$ cuando el CSV tiene 150, y la detección real regenerada es del 60 % (§16). **Cómo debe quedar:** las cinco características de la guía con métrica, umbral declarado en el protocolo previo, $n = 10$ réplicas reales tras el descarte, media, DE, IC95 % y contraste explícito con el umbral, calculados por `experimentos/iso25010.py` desde los crudos de §15 y §16 e incluidos en el manuscrito con `\input{tabla_iso.tex}`. **Cómo lo verifico:** `python experimentos/iso25010.py && git diff --exit-code experimentos/resultados/iso25010.csv Informe-E4_BCEL/tabla_iso.tex`.

§18 Manuscrito — afirmaciones que el repositorio desmiente

Qué encontré: compila a 29 páginas sin indefinidas (con 204 cajas desbordadas), pero el PDF versionado es del 12/09 y el `.tex` tiene tres commits posteriores: el PDF aún dice 12,265 cuatro veces, 30.31 y 40.96. Afirma cosas que el código no respalda: React con TypeScript, sesión sin `localStorage` con cookies `HttpOnly`, patrones GoF en flujo real, tres instancias Raft y particionado por rango, un flujo por issues, una tabla de revisión cruzada con validadores que no existen, cobertura del 90,7 % y Soporte en “Node.js Express” (es Spring Boot). Tiene dos secciones de fundamento teórico, filas ISO duplicadas, cifras inferenciales (medianas, U, A12) que no salen de ningún archivo y hashes inexistentes. De 43 entradas bibliográficas sólo se citan 14, de las que 8 tienen DOI y ninguna ISBN (se exigen ≥ 12), con dos pares duplicados y una cita mal atribuida. La matriz de trazabilidad cruza temas con archivos, no la evolución $E1 \rightarrow E4$, y cita rutas inexistentes. **Cómo debe quedar:** ninguna afirmación sin respaldo en el código; cifras incluidas desde tablas generadas por los guiones; ≥ 12 referencias citadas con DOI o ISBN y sin duplicados; sección de trazabilidad $E1 \rightarrow E4$ con commit y ruta de lo que cambió en cada entrega; PDF compilado y versionado en el mismo commit que su `.tex`, *después* de la última regeneración de resultados. **Cómo lo verifico:**

```
test "$(git log -1 --format=%H -- Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf)" = "$(git log -1 --format=%H --
↪ Informe-E4_BCEL/TA-PFC-E4_BCEL.tex)"
pdftotext Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf - | grep -cE "TypeScript|HttpOnly|12,265|90.7"      # 0
grep -c "\\\b\bitem" Informe-E4_BCEL/TA-PFC-E4_BCEL.bbl                                     # >= 12
```

§19 Amenazas a la validez y reflexión ética

Qué encontré: tres internas y dos externas, pero la mitigación por descarte no existe en el código, la de contención de CPU no tiene evidencia, las externas no tienen mitigación y no se trata la

amenaza principal: datos sintéticos y latencias generadas por fórmula. La reflexión ética tiene unas 85 palabras, cita el código ACM/IEEE-CS de 1999 y no aborda los problemas éticos del propio sistema: notas de menores enviadas sin autenticación a un servicio de IA externo, el respaldo con posibles datos personales y las credenciales publicadas. **Cómo debe quedar:** cada amenaza con una mitigación que apunte a un archivo verificable, más la validez de constructo por datos sintéticos; reflexión de al menos media página con el ACM Code of Ethics 2018 (1.3 honestidad, 1.6 privacidad, 2.9 seguridad) aplicado a esas decisiones concretas.

§20 Trabajo en equipo y autoría

Qué encontré: las tres actas, fechadas el 18/08, 28/08 y 03/09, entraron en un solo commit el 04/09 y contienen datos posteriores a su fecha (un tablero creado el 01/09, métricas del 31/08, una mediana de la corrida del 11/09) y afirman haber “auditado y validado” PR con cero revisiones; no hay actas del periodo de correcciones. La declaración de uso de IA existe en tres versiones incompatibles (una de ellas pide que “los demás integrantes completen”). 23 commits llevan el nombre de Emanuel Pino con el correo de Luna Mora, 16 de ellos creados en 37 segundos. Entre la evaluación del 12/09 y el corte, Bedón Viteri hizo el trabajo técnico de más peso (22 commits), Emanuel Pino 11, Castro López 7 cambios pequeños (uno de ellos introdujo los valores de reserva inseguros y otro las exclusiones de cobertura) y Luna Mora 2 (uno de ellos la contraseña literal de V18). **Cómo debe quedar:** nota en las tres actas que reconozca que se redactaron el 04/09 y corrija los anacronismos; actas nuevas versionadas el día de cada reunión del suspenso; una sola declaración de IA coherente entre README y manuscrito; identidades git correctas en cada equipo y aclaración de los commits de identidad mixta; cada ítem de esta guía con un responsable nominal y entregado en PR aprobados por otro integrante. **Como los cuatro rinden el examen, cada uno debe tener aporte técnico propio y verificable:** nadie aprueba con el trabajo de otra persona. **Cómo lo verifico:** `git log --diff-filter=A --format="%h%ad" --date=short -- docs/actas/` frente a la fecha de cada acta; `git shortlog -sne --no-merges 78b0d175..v4.0.0`; revisiones de cada PR en la API.

4. Orden de ejecución

§2a credenciales y datos personales (inmediato: retirar, rotar y purgar antes de cualquier otra reescritura) → §3 protección de `main` → §11 clúster y usuario sin privilegios y §12a cadena de auditoría → §4 capas y §5 patrones (en paralelo) → §6, §8 y §9 clientes → §7, §12b, §10 y §13 pruebas y CI → §14 observabilidad → §15 protocolo (commit) y campaña → §16 reproducibilidad → §17 ISO/IEC 25010 (con §12c rampa) → §18 y §19 con las cifras finales → §1, §2b, §2c y §20 → CHANGELOG, manifiestos y, lo último, la etiqueta anotada.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/acadtrace HEAD
git cat-file -t v4.0.0 && test "$(git rev-parse v4.0.0~{commit})" = "$(git rev-parse origin/main)" && test -f
↪ CHANGELOG.md
gitLeaks detect --source . --log-opts="--all"
for c in PortalAcademicoFacade ReporteNotasPeriodoPDF CalculoPromedioStrategy NotaPublicadaEvent; do echo $c
↪ $(grep -rl $c sga-principal/src/main/java | grep -vc "/$c.java"); done
python experimentos/run_experimentos.py --mode local --output-dir /tmp/a && sha256sum -c
↪ experimentos/SHA256SUMS
grep -n "<element>CLASS</element>|<include>" sga-principal/pom.xml microservicio-secretaria/backend/pom.xml
pdfToText Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf - | grep -cE "TypeScript|HttpOnly|12,265"
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.